

HITO 2 PARTE 2 DEL 3º TRIMESTRE DE PROGRAMACIÓN

Alejandro Cortés Díaz

Índice

SQL.....	2
Definicion de clases.....	3
Funcionamiento de clases.....	13
Funcionamiento código.....	14
Enlace a GitHub	17

SQL

```
Create table categoria (  
id_categoria int primary key,  
cat varchar (50)  
);
```

```
Create table pelicula (  
id_pelicula int primary key,  
Titulo varchar (50),  
Valoraciones varchar(50),  
id_categoria int,  
Lanzamiento date,  
Foreign key (Categoria) references Categoria(id_categoria)  
);
```

Definicion de clases

Main;

```
package Hito; // Importo el package donde se encuentran porque sin él me daba error al referenciar a
Controller
import java.sql.*; // Importación de herramientas sql
import java.sql.Date;
import java.util.*;

public class Main {

    // Creo instancia de Controller para usar sus métodos
    Controller controller = new Controller();
    // Creo instancia de scanner para insertar datos por consola
    Scanner scanner = new Scanner (System.in);

    // Instancia de la clase Main para utilizar el método menú
    public static void main (String[]args) {
        Main hito = new Main();
        hito.menu();
    }

    // En el menú doy las opciones posibles para ver películas via SQL y salir del programa
    public void menu() {
        int opcion = 0;

        //Con el do al menos una vez que ejecute el código para mostrar menú
        do {
            try {
                System.out.println("-----MENU-----")
                ----- \n 1 - Ver Películas\n 2 - Añadir Películas\n 3 - Eliminar Películas\n 4 - Modificar Películas\n 5
- Salir del programa ");

                //Instancia de scanner para escribir en consola
                opcion = Integer.parseInt(scanner.nextLine());

                switch(opcion) {

                    // según la opción ingresada se usa la función o se sale del programa
                    case 1:

                        controller.verPelículas();
                        break;

                    case 2:
                        Scanner scanner = new Scanner(System.in);

                        System.out.print("Introduce ID de la película: ");
                        int id = scanner.nextInt();
```

```

scanner.nextLine();

System.out.print("Introduce el título: ");
String titulo = scanner.nextLine();

System.out.print("Introduce las valoraciones: ");
String valoraciones = scanner.nextLine();

System.out.print("Introduce el ID de categoría : ");

// Aquí llamo a verCategoria que es simplemente un "select * from categoria" para que el
usuario sepa a qué categoría corresponde cada ID, y posteriormente escribirlo con conocimiento
controller.verCategoria();

int id_categoria = scanner.nextInt();
scanner.nextLine();

// Paso de String a Date para que coincida con el tipo de dato de la db
System.out.print("Introduce la fecha de lanzamiento (YYYY-MM-DD): ");
String fechaStr = scanner.nextLine();
Date lanzamiento = Date.valueOf(fechaStr);

controller.añadirPelicula(id, titulo, valoraciones, id_categoria, lanzamiento);
break;

case 3:

    // Pido el id que se desea eliminar tras mostrar las películas con sus ids
    Scanner scanner4 = new Scanner(System.in);

    System.out.print("Introduce el ID de la película que deseas eliminar: ");
    controller.verPeliculas();

    int id_Eliminar = scanner4.nextInt();

    controller.eliminarPelicula(id_Eliminar);
    break;

case 4:

    // Nuevamente pido datos para introducirlos como parámetros a la función de
modificación

    Scanner scanner3 = new Scanner(System.in);

    System.out.print("Introduce el ID de la película que quieres editar: ");
    int idEditar = scanner3.nextInt();
    scanner3.nextLine();

    System.out.print("Introduce el nuevo título: ");
    String newTitulo = scanner3.nextLine();

    System.out.print("Introduce las nuevas valoraciones: ");
    String newValoraciones = scanner3.nextLine();

    System.out.print("Introduce el nuevo ID de categoría: ");
    int newCategoria = scanner3.nextInt();
    scanner3.nextLine();

```

```

// Paso de String a Date para que coincida con el tipo de dato de la db

    System.out.print("Introduce la nueva fecha de lanzamiento (YYYY-MM-DD): ");
    String newFechaStr = scanner3.nextLine();
    Date newFecha = Date.valueOf(newFechaStr);

    controller.editarPelicula(idEditar, newTitulo, newValoraciones, newCategoria,
newFecha);

    break;

case 5:

    // Salgo del programa

    System.out.println("Saliendo del programa...");

    break;

//Valor por defecto si se introduce algo no registrado

default:
    System.out.println("Selecciona una opcion válida del menu");
}

// Registro de error
} catch (NumberFormatException e) {
    System.out.println("Por favor ingresa un número válido");
}

//Mientras que la opción elegida no sea cinco,, que sería para salir del programa
} while(opcion != 5);
}

}

```

```

1 package Hito; // Importo el package donde se encuentran porque sin él me daba error al referenciar a Controller
2 import java.sql.*; // Importación de herramientas sql []
3
4
5
6 public class Main {
7
8     // Creo instancia de Controller para usar sus métodos
9     Controller controller = new Controller();
10    // Creo instancia de scanner para insertar datos por consola
11    Scanner scanner = new Scanner(System.in);
12
13
14    // Instancia de la clase Main para utilizar el método menú
15    public static void main (String[] args) {
16        Main hito = new Main();
17        hito.menu();
18    }
19
20
21
22    // En el menú doy las opciones posibles para ver películas via SQL y salir del programa
23    public void menu() {
24        int opcion = 0;
25
26        // Con el do al menos una vez que ejecute el código para mostrar menú
27        do {
28            try {
29                System.out.println("-----MENU-----\n 1 - Ver Películas\n 2 - Añadir Películas\n 3 - Eliminar Películas\n 4 - Modificar Películas\n 5 - Salir del programa ");
30
31                // Instancia de scanner para escribir en consola
32                opcion = Integer.parseInt(scanner.nextLine());
33
34                switch(opcion) {
35
36                    // según la opción ingresada se usa la función o se sale del programa
37                    case 1:
38
39                        controller.verPelículas();
40                        break;
41
42                    case 2:
43                        Scanner scanner = new Scanner(System.in);
44
45                        System.out.print("Introduce ID de la película: ");
46                        int id = scanner.nextInt();
47                        scanner.nextLine();
48
49                        System.out.print("Introduce el título: ");
50                        String titulo = scanner.nextLine();
51
52                        System.out.print("Introduce las valoraciones: ");
53                        String valoraciones = scanner.nextLine();
54
55                        System.out.print("Introduce el ID de categoría: ");

```

```

// Aquí llamo a verCategoría que es simplemente un "select * from categoría" para que el usuario sepa a qué categoría corresponde cada ID, y posteriormente escribirlo con conocimiento
controller.verCategoría();

int id_categoria = scanner.nextInt();
scanner.nextLine();

// Paso de String a Date para que coincida con el tipo de dato de la db
System.out.print("Introduce la fecha de lanzamiento (YYYY-MM-DD): ");
String fechaStr = scanner.nextLine();
Date lanzamiento = Date.valueOf(fechaStr);

controller.añadirPelícula(id, titulo, valoraciones, id_categoria, lanzamiento);
break;

case 3:

// Pido el id que se desea eliminar tras mostrar las películas con sus ids
Scanner scanner4 = new Scanner(System.in);

System.out.print("Introduce el ID de la película que deseas eliminar: ");
controller.verPelículas();

int id_Eliminar = scanner4.nextInt();

controller.eliminarPelícula(id_Eliminar);
break;

case 4:

// Nuevamente pido datos para introducirlos como parámetros a la función de modificación
Scanner scanner3 = new Scanner(System.in);

System.out.print("Introduce el ID de la película que quieres editar: ");
int idEditar = scanner3.nextInt();
scanner3.nextLine();

System.out.print("Introduce el nuevo título: ");
String newTitulo = scanner3.nextLine();

System.out.print("Introduce las nuevas valoraciones: ");
String newValoraciones = scanner3.nextLine();

System.out.print("Introduce el nuevo ID de categoría: ");
int newCategoría = scanner3.nextInt();
scanner3.nextLine();

// Paso de String a Date para que coincida con el tipo de dato de la db

System.out.print("Introduce la nueva fecha de lanzamiento (YYYY-MM-DD): ");
String newFechaStr = scanner3.nextLine();
Date newFecha = Date.valueOf(newFechaStr);

```



```

    // Si la conexión falla mensaje error
    if (conexion == null) {
        System.out.println("No se pudo establecer conexión con la base de datos.");
        return;
    }
    try {
        // Creo la sentencia sql conectada a la db, y ejecuto la Query SQL
        Statement stmt = conexion.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM pelicula inner join categoria on
categoria.id_categoria = pelicula.id_categoria");

        //Mientras siga encontrando resultados, que los muestre
        while (rs.next()) {
            System.out.println("ID: " + rs.getInt("id_pelicula") + " Titulo: " + rs.getString("titulo")+ "
Valoraciones: "+rs.getString("valoraciones")+ " ID_Categoria: " + rs.getInt("id_categoria")+ " Fecha de
extremo: "+rs.getDate("lanzamiento")+"\n" );
        }

        //Sino, mensaje de error
    } catch (SQLException e) {
        System.out.println("Error al recuperar películas: " + e.getMessage());
    }
}

// Defino función para ver las categorías disponibles
public void verCategoria() {

    // Si la conexión falla mensaje error
    if (conexion == null) {
        System.out.println("No se pudo establecer conexión con la base de datos.");
        return;
    }

    try {
        // Creo la sentencia sql conectada a la db, y ejecuto la Query SQL
        Statement stmt = conexion.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM categoria ");

        System.out.println("Mostrando Categorías disponibles \n");

        //Mientras siga encontrando resultados, que los muestre
        while (rs.next()) {
            System.out.println("ID: " + rs.getInt("id_categoria") + " Categoría: " + rs.getString("cat")+"\n" );
        }

        //Sino, mensaje de error
    } catch (SQLException e) {
        System.out.println("Error al recuperar categorias: " + e.getMessage());
    }
}

// Función para añadir película
public void añadirPelicula(int id_pelicula, String titulo, String valoraciones, int id_categoria, Date
lanzamiento) {

    //Recojo que pueda fallar la conexión
    if (conexion == null) {

```

```

        System.out.println("No se pudo establecer conexión con la base de datos.");
        return;
    }

    // Ejecuto la sentencia SQL tras la conexión para comprobar si existe el id
    try {
        PreparedStatement checkStmt = conexion.prepareStatement("SELECT * FROM pelicula WHERE
id_pelicula = ?");
        checkStmt.setInt(1, id_pelicula);
        ResultSet rs = checkStmt.executeQuery();

        // En el caso de que detecte un resultado con el id pasado por parámetro, me informa de que no
        podría hacerlo por la repetición de ids
        if (rs.next()) {
            System.out.println("Ya existe una película con ese ID.");
            return;
        }

        //Sino, preparo la sentencia de insert into con los datos necesarios para ingresar la nueva película a
        la tabla y ejecuto

        PreparedStatement insertStmt = conexion.prepareStatement(
            "INSERT INTO pelicula (id_pelicula, titulo, valoraciones, id_categoria, lanzamiento) VALUES (?, ?,
?, ?, ?)");
        insertStmt.setInt(1, id_pelicula);
        insertStmt.setString(2, titulo);
        insertStmt.setString(3, valoraciones);
        insertStmt.setInt(4, id_categoria);
        insertStmt.setDate(5, lanzamiento);

        insertStmt.executeUpdate();
        System.out.println("Película añadida correctamente.");
    } catch (SQLException e) {
        //Sino, mensaje de error
        System.out.println("Error al añadir la película: " + e.getMessage());
    }
}

// Función para editar película
public void editarPelicula(int id_pelicula, String nuevoTitulo, String nuevasValoraciones, int
nuevaCategoria, Date nuevaFecha) {

    //Recojo que falle conexión
    if (conexion == null) {
        System.out.println("No se pudo establecer conexión con la base de datos.");
        return;
    }

    //Sino, preparo la sentencia del update con los datos necesarios para modificar el contenido de la
    tabla y ejecuto

    try {
        PreparedStatement updateStmt = conexion.prepareStatement(
            "UPDATE pelicula SET titulo = ?, valoraciones = ?, id_categoria = ?, lanzamiento = ? WHERE
id_pelicula = ?");
        updateStmt.setString(1, nuevoTitulo);
        updateStmt.setString(2, nuevasValoraciones);
        updateStmt.setInt(3, nuevaCategoria);
    }
}

```

```
updateStmt.setDate(4, nuevaFecha);
updateStmt.setInt(5, id_pelicula);
```

//Aquí si se ejecuta la sentencia update, fila resulta >0 y por tanto se habrían modificado correctamente los datos, sino, se comunica mediante mensaje de error que no se estarían referenciando correctamente los datos

```
int filas = updateStmt.executeUpdate();
if (filas > 0) {
    System.out.println("Película actualizada correctamente.");
} else {
    System.out.println("Algún/Algunos datos no fueron correctos en la ejecución.");
}
} catch (SQLException e) {
    // mensaje de excepción por fallo
    System.out.println("Error al editar la película: " + e.getMessage());
}
}
```

// Función para eliminar película

```
public void eliminarPelicula(int id_pelicula) {
```

//Recojo que falle conexión

```
if (conexion == null) {
    System.out.println("No se pudo establecer conexión con la base de datos.");
    return;
}
```

// Preparo la sentencia delete para eliminar película por id, y ejecuto

```
try {
    PreparedStatement deleteStmt = conexion.prepareStatement("DELETE FROM pelicula WHERE
id_pelicula = ?");
    deleteStmt.setInt(1, id_pelicula);
```

// Si no ha habido eliminación el valor de "filas" se mantiene como 0, por lo que avisaría por consola, de lo contrario, simplemente se ejecuta y elimina la película, con su correspondiente mensaje de éxito

```
int filas = deleteStmt.executeUpdate();
if (filas > 0) {
    System.out.println("Película eliminada correctamente.");
} else {
    System.out.println("No se encontró una película con ese ID.");
}
```

//Mensaje de excepción si hay error

```
} catch (SQLException e) {
    System.out.println("Error al eliminar la película: " + e.getMessage());
}
}
```

```
}
```

```

package Hito; // Importo el package donde se encuentran porque sin él me daba error al referenciar a Controller
import java.sql.*; // Importación de herramientas sql

// Clase Controller que se encarga de preciar el puerto, el nombre de la db, el user, y la password de la base de datos al inicio, y crea un atributo conexion
public class Controller {

    private final String url = "jdbc:mysql://localhost:3306/cine_alejandro_cortés_díaz";
    private final String usuario = "root";
    private final String contraseña = "";

    private Connection conexion;

    // En el constructor conecto con db directamente
    public Controller() {
        try {
            this.conexion = DriverManager.getConnection(url, usuario, contraseña);
        } catch (SQLException e) {
            System.out.println("Error al conectar a la base de datos: " + e.getMessage());
        }
    }

    // Defino el método para ver las películas almacenadas
    public void verPelículas() {
        // Si la conexión falla mensaje error
        if (conexion == null) {
            System.out.println("No se pudo establecer conexión con la base de datos.");
            return;
        }
        try {
            // Creo la sentencia sql conectada a la db, y ejecuto la Query SQL
            Statement stmt = conexion.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM pelicula inner join categoria on categoria.id_categoria = pelicula.id_categoria");

            // Mientras siga encontrando resultados, que los muestre
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id_pelicula") + " Título: " + rs.getString("titulo") + " Valoraciones: " + rs.getString("valoraciones") + " ID_Categoria: " + rs.getInt("id_categoria") + " Fecha de estreno: " + rs.getDate("lanzamiento") + "\n");
            }

            // Si no, mensaje de error
        } catch (SQLException e) {
            System.out.println("Error al recuperar películas: " + e.getMessage());
        }
    }

    // Defino función para ver las categorías disponibles
    public void verCategorias() {
        // Si la conexión falla mensaje error
        if (conexion == null) {
            System.out.println("No se pudo establecer conexión con la base de datos.");
            return;
        }
        try {
            // Creo la sentencia sql conectada a la db, y ejecuto la Query SQL
            Statement stmt = conexion.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM categoria");

            System.out.println("Mostrando Categorías disponibles \n");

            // Mientras siga encontrando resultados, que los muestre
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt("id_categoria") + " Categoría: " + rs.getString("cat") + "\n");
            }

            // Si no, mensaje de error
        } catch (SQLException e) {
            System.out.println("Error al recuperar categorías: " + e.getMessage());
        }
    }

    // Función para añadir película
    public void añadirPelícula(int id_pelicula, String titulo, String valoraciones, int id_categoria, Date lanzamiento) {
        // Recibo que pueda fallar la conexión
        if (conexion == null) {
            System.out.println("No se pudo establecer conexión con la base de datos.");
            return;
        }

        // Ejecuto la sentencia SQL tras la conexión para comprobar si existe el id
        try {
            PreparedStatement checkStmt = conexion.prepareStatement("SELECT * FROM pelicula WHERE id_pelicula = ?");
            checkStmt.setInt(1, id_pelicula);
            ResultSet rs = checkStmt.executeQuery();

            // En el caso de que detecte un resultado con el id pasado por parámetro, me informa de que no podría hacerlo por la repetición de ids
            if (rs.next()) {
                System.out.println("Ya existe una película con ese ID.");
                return;
            }

            // Si no, preparo la sentencia de insert into con los datos necesarios para ingresar la nueva película a la tabla y ejecuto
            PreparedStatement insertStmt = conexion.prepareStatement(
                "INSERT INTO pelicula (id_pelicula, titulo, valoraciones, id_categoria, lanzamiento) VALUES (?, ?, ?, ?, ?)");
            insertStmt.setInt(1, id_pelicula);
            insertStmt.setString(2, titulo);
            insertStmt.setString(3, valoraciones);
            insertStmt.setInt(4, id_categoria);
            insertStmt.setDate(5, lanzamiento);
        }
    }
}

```


Funcionamiento de clases

- Main; En main se realizan las importaciones de las funciones sql necesarias para ver las películas, añadirlas, eliminarlas y modificarlas. Mostramos un menú con las opciones posibles. Según la opción elegida, se usará la función comentada, alojada en la clase "Controller.java" o se saldrá del programa. Cada función con su correspondiente registro de errores.
- Controller; En este archivo encontramos la conexión a la db con los datos necesarios, nos conectamos a la base de datos insertando los atributos previamente descritos, y se definen las siguientes funciones;
- "verPelículas()", que, en el caso de que la conexión no haya retornado "null", pasará a aprovecharla para ejecutar una sentencia "SELECT" que nos permita ver las películas almacenadas en la base de datos. Cada función con su correspondiente registro de errores.
- "añadirPelícula()", comprueba que la conexión a la base de datos fue efectiva y procede a comprobar si el id pasado como parámetro coincide con alguno ya existente en la base de datos, en cuyo caso lo comunica. De no ser este el caso, prepara una sentencia SQL que contenga los valores que se deseen introducir, previamente establecidos en la llamada de la función, y ejecuta.
- "verCategoria()", únicamente existe para que, cuando se desee insertar alguna película, se sepan a qué categorías se estaría refiriendo el usuario, según el número de identificación que estuviera eligiendo. Es un select que opera igual que la función "verPelículas" pero con la tabla categoría.
- "eliminarPelícula()", teniendo como parámetro el id de la película que se desee eliminar, prepara la sentencia "delete" que se empleará para la eliminación de la película por id, en el caso de que no se efectúe como se espera, se comunica a través de la consola porque no se estaría encontrando el id que el usuario facilitase. De lo contrario, si "filas", que se crea para comprobar que se está ejecutando correctamente la sentencia SQL, pasa a ser 1, porque operó como se esperaba, se comunica por consola el mensaje de éxito tras eliminar la fila deseada.
- "editarPelícula()", prepara la sentencia SQL para modificar una película acorde al id que se habría pasado anteriormente al programa como parámetro, trata de ejecutar la sentencia, y en el caso de que "filas", con la misma función que en el caso anterior, marque 1 por la exitosa edición, lo comunica por consola, así como el caso opuesto donde haya un error en la comprobación de los datos

Funcionamiento código

Ver Películas;

```
5 - Salir del programa
1
ID: 101 Titulo: Matrix Valoraciones: Excelente pelicula de acción y ciencia ficción ID_Categoria: 3 Fecha de estreno: 1999-03-31
ID: 102 Titulo: Toy Story Valoraciones: Una historia divertida y entrañable para todas las edades ID_Categoria: 5 Fecha de estreno: 1995-11-22
ID: 103 Titulo: Titanic Valoraciones: Romance y tragedia con gran producción ID_Categoria: 4 Fecha de estreno: 1997-12-19
ID: 104 Titulo: John Wick Valoraciones: Acción intensa y coreografías espectaculares ID_Categoria: 1 Fecha de estreno: 2014-10-24
ID: 105 Titulo: La Máscara Valoraciones: Comedia con efectos innovadores y gran actuación ID_Categoria: 2 Fecha de estreno: 1994-07-29

-----MENU-----
1 - Ver Películas
2 - Añadir Películas
3 - Eliminar Películas
4 - Modificar Películas
5 - Salir del programa
```

Añadir Películas (Aquí también se ve verCategoria());

```
2
ID: 101 Titulo: Matrix Valoraciones: Excelente pelicula de acción y ciencia ficción ID_Categoria: 3 Fecha de estreno: 1999-03-31
ID: 102 Titulo: Toy Story Valoraciones: Una historia divertida y entrañable para todas las edades ID_Categoria: 5 Fecha de estreno: 1995-11-22
ID: 103 Titulo: Titanic Valoraciones: Romance y tragedia con gran producción ID_Categoria: 4 Fecha de estreno: 1997-12-19
ID: 104 Titulo: John Wick Valoraciones: Acción intensa y coreografías espectaculares ID_Categoria: 1 Fecha de estreno: 2014-10-24
ID: 105 Titulo: La Máscara Valoraciones: Comedia con efectos innovadores y gran actuación ID_Categoria: 2 Fecha de estreno: 1994-07-29

Introduce ID de la película: 106
Introduce el título: Prueba hito
Introduce las valoraciones: Hito CRUD
Introduce el ID de categoría : Mostrando Categorías disponibles

ID: 1 Categoría: Acción
ID: 2 Categoría: Comedia
ID: 3 Categoría: Ciencia Ficción
ID: 4 Categoría: Drama
ID: 5 Categoría: Animación

4
Introduce la fecha de lanzamiento (YYYY-MM-DD): 2025-05-07
Pelicula añadida correctamente.

1
ID: 101 Titulo: Matrix Valoraciones: Excelente pelicula de acción y ciencia ficción ID_Categoria: 3 Fecha de estreno: 1999-03-31
ID: 102 Titulo: Toy Story Valoraciones: Una historia divertida y entrañable para todas las edades ID_Categoria: 5 Fecha de estreno: 1995-11-22
ID: 103 Titulo: Titanic Valoraciones: Romance y tragedia con gran producción ID_Categoria: 4 Fecha de estreno: 1997-12-19
ID: 104 Titulo: John Wick Valoraciones: Acción intensa y coreografías espectaculares ID_Categoria: 1 Fecha de estreno: 2014-10-24
ID: 105 Titulo: La Máscara Valoraciones: Comedia con efectos innovadores y gran actuación ID_Categoria: 2 Fecha de estreno: 1994-07-29
ID: 106 Titulo: Prueba hito Valoraciones: Hito CRUD ID_Categoria: 4 Fecha de estreno: 2025-05-07
```

editarPelicula;

```

4
ID: 101 Titulo: Matrix Valoraciones: Excelente pelicula de acción y ciencia ficción ID_Categoria: 3 Fecha de estreno: 1999-03-31
ID: 102 Titulo: Toy Story Valoraciones: Una historia divertida y entrañable para todas las edades ID_Categoria: 5 Fecha de estreno: 1995-11-22
ID: 103 Titulo: Titanic Valoraciones: Romance y tragedia con gran producción ID_Categoria: 4 Fecha de estreno: 1997-12-19
ID: 104 Titulo: John Wick Valoraciones: Acción intensa y coreografías espectaculares ID_Categoria: 1 Fecha de estreno: 2014-10-24
ID: 105 Titulo: La Máscara Valoraciones: Comedia con efectos innovadores y gran actuación ID_Categoria: 2 Fecha de estreno: 1994-07-29
ID: 106 Titulo: Prueba hito Valoraciones: Hito CRUD ID_Categoria: 4 Fecha de estreno: 2025-05-07

Introduce el ID de la película que quieres editar: 106
Introduce el nuevo título: Prueba de Hito mejorado
Introduce las nuevas valoraciones: Me gusta el morado
Mostrando Categorías disponibles

ID: 1 Categoría: Acción
ID: 2 Categoría: Comedia
ID: 3 Categoría: Ciencia Ficción
ID: 4 Categoría: Drama
ID: 5 Categoría: Animación

Introduce el nuevo ID de categoría: 1
Introduce la nueva fecha de lanzamiento (YYYY-MM-DD): 2025-05-08
Película actualizada correctamente.

```

```

1
ID: 101 Titulo: Matrix Valoraciones: Excelente pelicula de acción y ciencia ficción ID_Categoria: 3 Fecha de estreno: 1999-03-31
ID: 102 Titulo: Toy Story Valoraciones: Una historia divertida y entrañable para todas las edades ID_Categoria: 5 Fecha de estreno: 1995-11-22
ID: 103 Titulo: Titanic Valoraciones: Romance y tragedia con gran producción ID_Categoria: 4 Fecha de estreno: 1997-12-19
ID: 104 Titulo: John Wick Valoraciones: Acción intensa y coreografías espectaculares ID_Categoria: 1 Fecha de estreno: 2014-10-24
ID: 105 Titulo: La Máscara Valoraciones: Comedia con efectos innovadores y gran actuación ID_Categoria: 2 Fecha de estreno: 1994-07-29
ID: 106 Titulo: Prueba de Hito mejorado Valoraciones: Me gusta el morado ID_Categoria: 1 Fecha de estreno: 2025-05-08

```

eliminarPelicula();

```

3
Introduce el ID de la película que deseas eliminar:
ID: 101 Titulo: Matrix Valoraciones: Excelente pelicula de acción y ciencia ficción ID_Categoria: 3 Fecha de estreno: 1999-03-31
ID: 102 Titulo: Toy Story Valoraciones: Una historia divertida y entrañable para todas las edades ID_Categoria: 5 Fecha de estreno: 1995-11-22
ID: 103 Titulo: Titanic Valoraciones: Romance y tragedia con gran producción ID_Categoria: 4 Fecha de estreno: 1997-12-19
ID: 104 Titulo: John Wick Valoraciones: Acción intensa y coreografías espectaculares ID_Categoria: 1 Fecha de estreno: 2014-10-24
ID: 105 Titulo: La Máscara Valoraciones: Comedia con efectos innovadores y gran actuación ID_Categoria: 2 Fecha de estreno: 1994-07-29
ID: 106 Titulo: Prueba de Hito mejorado Valoraciones: Me gusta el morado ID_Categoria: 1 Fecha de estreno: 2025-05-08

106
Película eliminada correctamente.

PELICULA ELIMINADA CORRECTAMENTE.
-----MENU-----
1 - Ver Películas
2 - Añadir Películas
3 - Eliminar Películas
4 - Modificar Películas
5 - Salir del programa

1
ID: 101 Titulo: Matrix Valoraciones: Excelente pelicula de acción y ciencia ficción ID_Categoria: 3 Fecha de estreno: 1999-03-31
ID: 102 Titulo: Toy Story Valoraciones: Una historia divertida y entrañable para todas las edades ID_Categoria: 5 Fecha de estreno: 1995-11-22
ID: 103 Titulo: Titanic Valoraciones: Romance y tragedia con gran producción ID_Categoria: 4 Fecha de estreno: 1997-12-19
ID: 104 Titulo: John Wick Valoraciones: Acción intensa y coreografías espectaculares ID_Categoria: 1 Fecha de estreno: 2014-10-24
ID: 105 Titulo: La Máscara Valoraciones: Comedia con efectos innovadores y gran actuación ID_Categoria: 2 Fecha de estreno: 1994-07-29

```

Salir del programa;

MENU

- 1 - Ver Peliculas
- 2 - Añadir Peliculas
- 3 - Eliminar Peliculas
- 4 - Modificar Peliculas
- 5 - Salir del programa

5
Saliendo del programa...

Enlace a GitHub

<https://github.com/Cortes-cmd/Programacion.git>